

Universidade de Brasília

Faculdade UnB Gama

PSPD – Programação para Sistemas Paralelos e Distribuídos

Prof.: Fernando W. Cruz

Turma 2026.1

Relatório do Projeto de Pesquisa

Monitoramento e Observabilidade de Aplicações em Clusters Kubernetes

Grupo 5

Fábio Gabriel
Lucas Meireles
Vinicius Alves

Brasília, julho de 2026

1. Introdução

Este relatório documenta o desenvolvimento do projeto de pesquisa proposto na disciplina de Programação para Sistemas Paralelos e Distribuídos (PSPD), cujo objetivo é explorar estratégias de monitoramento e observabilidade de aplicações baseadas em microsserviços em ambiente Kubernetes, com foco em métricas de desempenho sob diferentes cenários de carga e escalabilidade.

Para atender à especificação, o grupo projetou e implementou uma aplicação baseada em microsserviços que simula o prontuário eletrônico de um Hospital Universitário, seguindo o padrão HL7/FHIR para representação dos dados clínicos. A aplicação é composta por um frontend, uma API Gateway e três microsserviços internos (Authorization, Patient Data e Data Transform), comunicando-se via gRPC, com autenticação OAuth2/OpenID Connect via Keycloak e persistência em PostgreSQL.

A aplicação foi implantada em um cluster Kubernetes (1 nó mestre e 3 nós workers) e submetida a testes de carga progressivos (10, 50, 100, 500 e 1000 usuários virtuais), em três cenários distintos: réplica única (baseline), escalabilidade horizontal manual (3 réplicas) e autoscaling via Horizontal Pod Autoscaler (HPA). Toda a instrumentação foi observada por meio de uma stack de monitoramento Prometheus + Grafana, com métricas de infraestrutura e de negócio.

Este documento está organizado da seguinte forma: a Seção 2 descreve a metodologia, a arquitetura da aplicação e a organização do grupo; a Seção 3 relata a experiência de montagem do cluster Kubernetes; a Seção 4 apresenta cada uma das cinco fases do projeto (validação funcional, testes de carga, escalabilidade horizontal, autoscaling e observabilidade) com os respectivos resultados; a Seção 5 traz os erros encontrados durante o desenvolvimento e as correções aplicadas; a Seção 6 destaca as funcionalidades adicionais não solicitadas pela especificação; a Seção 7 conclui o trabalho; a Seção 8 traz as conclusões individuais de cada integrante; e a Seção 9 lista as referências utilizadas.

2. Metodologia e Arquitetura

2.1 Arquitetura da Aplicação

A aplicação segue a arquitetura exigida pela especificação (Figuras 1 e 2 do documento de proposta): uma API Gateway recebe requisições HTTP REST, valida o token JWT e atua como cliente gRPC assíncrono para três microsserviços internos, que por sua vez consultam o banco de dados e transformam os dados no padrão HL7/FHIR antes de devolvê-los ao usuário. A tabela a seguir resume os componentes e as tecnologias escolhidas.

Componente	Tecnologia	Justificativa
Frontend	HTML/JS + Nginx	SPA leve para demonstração; Nginx atua como proxy reverso do Gateway, eliminando problemas de CORS.
API Gateway	Python FastAPI + Uvicorn	Alto desempenho em operações assíncronas (I/O bound); valida o JWT localmente via chave pública (JWKS) do Keycloak, sem chamada de rede extra por requisição; expõe Swagger UI em /docs.

Componente	Tecnologia	Justificativa
Comunicação interna	gRPC (HTTP/2 + Protobuf)	Exigência da especificação; contrato estrito (hospital.proto), payload binário compacto e menor latência em cenários de alto tráfego.
Authorization Service	gRPC (Python)	Verifica permissões do usuário a partir do JWT e das tabelas de vínculo, determinando o nível de acesso (FULL, PARTIAL, ANONYMIZED, AGGREGATED).
Patient Data Service	gRPC (Python)	Executa as consultas SQL sobre o pseudo-prontuário eletrônico e monta os dados clínicos brutos.
Data Transform Service	gRPC (Python)	Aplica anonimização, agregação e conversão dos dados para o padrão FHIR antes da entrega ao usuário.
Identidade	Keycloak (OAuth2/OIDC)	Padrão de mercado para IAM; realm hospital versionado em JSON, importado automaticamente na subida.
Banco de dados	PostgreSQL	Banco relacional robusto; massa de dados sintética gerada por script Python determinístico (200 pacientes, ~5.400 registros clínicos).
Cluster	kind (Kubernetes in Docker)	Permite criar um cluster multinó localmente (1 control-plane + 3 workers) usando containers Docker como nós.
Observabilidade	kube-prometheus-stack (Helm)	Instala Prometheus Operator, Alertmanager, Grafana e exporters de uma só vez; ServiceMonitors customizados coletam métricas de negócio.
Teste de carga	k6 (principal) e Locust (alternativa)	k6 permite parametrizar VUs por variável de ambiente e exportar sumário JSON; Locust oferece painel web de acompanhamento.

O fluxo de uma requisição típica é: Browser → Nginx (frontend) → API Gateway (valida JWT) → Authorization Service (decide ALLOW/DENY e o nível de acesso) → Patient Data Service (consulta SQL) → Data Transform Service (converte para FHIR e anonimiza conforme o nível) → resposta JSON FHIR ao navegador.

2.2 Regras de Negócio e Autorização (RBAC)

O sistema implementa quatro níveis de acesso, de acordo com o perfil do usuário autenticado:

Perfil	Escopo de acesso	Nível de dado retornado
Médico (MEDICO)	Somente pacientes vinculados a ele	FULL — dados completos, incluindo CPF, CNS e data de nascimento exata.
Estagiário (ESTAGIARIO)	Pacientes do médico supervisor	PARTIAL — sem CPF/CNS; data de nascimento substituída por faixa etária.
Pesquisador (PESQUISADOR)	Coortes de projetos aprovados e vigentes	ANONYMIZED (IDs em hash, sem nomes) para consultas de coorte, e AGGREGATED (recurso FHIR MeasureReport) para estatísticas.
Qualquer outro caso	—	DENY (HTTP 403).

2.3 Estrutura de Arquivos do Projeto

Diretório	Conteúdo
proto/	Contrato gRPC único (hospital.proto), compartilhado pelos quatro serviços.
services/	Código-fonte, Dockerfile e requirements de cada serviço (api-gateway, authorization, patient-data, data-transform, frontend).

Diretório	Conteúdo
database/	01-schema.sql, 02-seed.sql (massa de dados pronta) e generate_seed.py (gerador da massa sintética).
keycloak/	realm-hospital.json, com roles, client e usuários de teste.
k8s/	Manifests numerados: cluster kind, namespace/secret, postgres, keycloak, microsserviços, gateway/frontend, HPA e ServiceMonitors.
grafana/	dashboard-hospital.json, com 10 painéis prontos para importação.
load-tests/	k6-smoke.js, k6-load.js, locustfile.py, run-all-scenarios.sh.
scripts/	Automação: setup-cluster.sh, build-images.sh, deploy-all.sh, scale.sh, collect-metrics.sh, validate-functional.sh.
docs/	Guias 01 a 05 (visão geral, validação funcional, cluster, testes de carga, observabilidade).

2.4 Organização do Grupo e Encontros

Devido às dificuldades de agenda no encerramento do semestr, coincidindo com a entrega de outros projetos e avaliações, o grupo optou por manter a comunicação e o desenvolvimento de forma predominantemente assíncrona, distribuindo as responsabilidades por área de atuação: desenvolvimento dos serviços e contrato gRPC, documentação de reprodução do ambiente, operação/deploy no cluster e execução dos testes de carga.

3. Experiência de Montagem do Kubernetes em Modo Cluster

A infraestrutura Kubernetes utilizada para a implantação final foi disponibilizada pela disciplina (cluster remoto compartilhado), o que posicionou o grupo predominantemente no papel de Operação e Deploy (DevOps), em vez de provisionamento da infraestrutura base. Adicionalmente, para a fase de validação local (Fase a) e para experimentação própria, o grupo também utilizou o kind para criar um cluster local com a topologia exigida (1 control-plane e 3 workers), documentada nos guias técnicos do projeto.

3.1 Criação do Cluster Local (kind)

O cluster local é criado a partir do arquivo k8s/kind-cluster.yaml, com a topologia de 1 nó control-plane e 3 nós workers. Um script automatizado (scripts/setup-cluster.sh) executa as seguintes etapas:

- Criação do cluster kind com a topologia exigida;
- Instalação do metrics-server, necessário para o autoscaling (HPA);
- Instalação da stack de monitoramento kube-prometheus-stack via Helm;
- Instalação do Kubernetes Dashboard.

```
./scripts/setup-cluster.sh
kubectl get nodes # deve exibir 1 control-plane e 3 workers
```

Em seguida, as imagens Docker de cada serviço são construídas e carregadas diretamente nos nós do cluster (scripts/build-images.sh --kind), evitando a dependência de um registry externo, e os manifests Kubernetes são aplicados em sequência (scripts/deploy-all.sh), que aguarda todos os Pods atingirem o estado Running antes de finalizar.

3.2 Acesso ao Cluster Remoto da Disciplina

Para a implantação no cluster remoto fornecido pela disciplina, o grupo trabalhou isolado no namespace grupo-5, autenticando-se por meio do arquivo kubeconfig-grupo-5.yaml fornecido. Toda injeção de credenciais sensíveis (usuário/senha do PostgreSQL, client secret do Keycloak) foi feita via Secrets do Kubernetes.

O principal desafio real de administração enfrentado foi a ResourceQuota restrita do namespace (limits.cpu: 6). Como cada Pod reservava uma cota teórica de CPU nos manifests originais, o Kubernetes inicialmente impediu escalar todos os serviços simultaneamente, mantendo, por exemplo, o api-gateway limitado a uma única réplica por falta de cota disponível. A solução adotada foi medir o consumo real de CPU de cada Pod no Grafana (inferior a 0,2 vCPU em todos os casos) e reduzir proporcionalmente os limits/requests declarados nos manifests YAML, liberando cota suficiente para escalar todos os serviços — incluindo o API Gateway — para 3 réplicas simultâneas.

4. Fases do Projeto e Resultados

4.1 Fase A — Validação Funcional

Antes de orquestrar a aplicação no Kubernetes, a arquitetura e as regras de negócio foram validadas em ambiente simplificado via Docker Compose, subindo os 7 containers da aplicação (postgres, keycloak, authorization, patient-data, data-transform, api-gateway e frontend).

```
for s in api-gateway authorization patient-data data-transform; do cp
proto/hospital.proto services/$s/; done
docker compose up -d --build
./scripts/validate-functional.sh
```

O script de validação automatizada executa uma bateria de verificações cobrindo: login de todos os perfis no Keycloak; rejeição de senha incorreta; acesso FULL do médico (CPF presente); DENY (HTTP 403) para paciente de outro médico; anonimização PARTIAL do estagiário (CPF ausente, faixa etária presente); coorte ANONYMIZED do pesquisador (IDs em hash, sem nomes/cidades); estatísticas AGGREGATED (recurso FHIR MeasureReport); DENY para coorte sem projeto aprovado ou com projeto expirado; e rejeição de requisições sem token ou com token inválido (HTTP 401).

Resultado obtido na execução de referência: 31 verificações PASS / 0 FAIL.

A validação também foi repetida manualmente pela interface web (<http://localhost:8081>), confirmando visualmente que: (i) o médico med.cardoso via CPF e data de nascimento completa no resumo clínico; (ii) o estagiário est.oliveira via o mesmo resumo sem o CPF e com a data de nascimento substituída por uma faixa etária; (iii) a tentativa de acessar um paciente de outro médico retornava 403; e (iv) o pesquisador pesq.ramos via a coorte sem nomes e com identificadores em hash.

Ao final desta fase, a mesma validação automatizada foi repetida contra o cluster Kubernetes, para confirmar que o comportamento funcional se manteve idêntico após a implantação em contêineres orquestrados:

```
BASE_URL=http://localhost:30800 ./scripts/validate-functional.sh
```

A aplicação implantada com uma réplica de cada microserviço e uma instância do PostgreSQL funcionou conforme esperado: o acesso com o usuário med.cardoso retornou todos os dados (FULL), enquanto o usuário est.ferreira teve o CPF removido da resposta JSON (PARTIAL), confirmando a correta aplicação das regras de anonimização também no ambiente de cluster.

4.2 Fase B — Testes de Carga (Baseline, 1 Réplica)

Os testes de carga foram realizados com a ferramenta k6, simulando 10, 50, 100, 500 e 1000 usuários virtuais (VUs) simultâneos. O script de carga (load-tests/k6-load.js) simula o comportamento real do sistema: os VUs possuem perfis ponderados (médicos realizam mais requisições que pesquisadores), fazem login para obter o JWT, executam chamadas aleatórias (lista de pacientes, resumos clínicos, coortes) e aguardam um think time de 0,5 a 2,5 s entre requisições, simulando a leitura da tela por um usuário humano.

Com 1 réplica de cada microserviço (10 workers gRPC configurados e pool de conexões limitado a 15 no PostgreSQL), os resultados evidenciaram claramente os gargalos do sistema conforme a carga aumentava:

VUs	Latência média	Latência p95	Throughput	Taxa de erro
100	233,5 ms	886,4 ms	48,4 req/s	0,00%
500	2193,1 ms	4745,8 ms	70,4 req/s	0,19%
1000	2464,3 ms	10854,5 ms	—	79,86%

Com 100 VUs, a aplicação suportou bem a carga, com consumo de CPU e memória baixo nos painéis do Grafana. Com 500 VUs observou-se o início da formação de filas nos microserviços; a pequena taxa de erro registrada não decorreu de falhas da aplicação (que continuou respondendo HTTP 200), mas de timeouts de conexão TCP, indicando que uma única réplica satura o número de conexões simultâneas suportadas pelo Gateway. Com 1000 VUs, o sistema com 1 réplica entrou em colapso de conexões de rede: o NGINX Ingress Controller e a camada de socket TCP passaram a negar conexões por timeout, atingindo 79,86% de taxa de erro — estabelecendo o limite de saturação da configuração baseline.

Evidências — Cenário Baseline (1 Réplica)

Console k6 — 100 VUs: throughput 48,4 req/s, latência média 233,5 ms, p95 886,4 ms, 0,00% de erro.

```
Quantos usuários simultâneos você quer simular? (ex: 10, 50, 100): 100
Rodando teste com 100 usuários...

      .--.
     /  \
    /    \
   /      \
  /        \
 /          \
/            \
 \          /
  \        /
   \      /
    \    /
     \  /
      --

execution: local
script: load-tests\k6-load.js
output: -

scenarios: (100.00%) 1 scenario, 100 max VUs, 2m15s max duration (incl. graceful stop):
            * default: Up to 100 looping VUs for 1m45s over 3 stages (gracefulRampDown: 30s, gracefulStop: 30s)

INFO[0107]
===== RESUMO DO CENÁRIO =====
VUs=100 reqs=5157 throughput=48.4/s lat_avg=233.5ms lat_p95=886.4ms err=0.00%
===== source=console =====

running (1m46.6s), 000/100 VUs, 4757 complete and 0 interrupted iterations
default ✓ [=====] 000/100 VUs 1m45s
Teste finalizado!
Pressione qualquer tecla para sair...
```

Grafana — uso de CPU por pod durante o teste de 100 VUs. O consumo permanece bem abaixo da cota (linhas tracejadas em 3 e 6 vCPU).



Grafana — uso de memória e rede por pod durante o teste de 100 VUs (1 réplica de cada serviço).

Memory Quota							
Workload	Type	Running Pods	Memory Usage	Memory Requests	Memory Requests %	Memory Limits	Memory Limits %
data-transform	deployment	1	26.9 MiB	128 MiB	21.0%	256 MiB	10.5%
frontend	deployment	1	10.0 MiB	32 MiB	31.4%	64 MiB	15.7%
api-gateway	deployment	1	138 MiB	192 MiB	72.1%	384 MiB	36.1%
authorization	deployment	1	25.3 MiB	128 MiB	19.7%	256 MiB	9.87%
patient-data	deployment	1	31.3 MiB	128 MiB	24.4%	256 MiB	12.2%

Current Network Usage						
Workload	Current Receive Bandwidth	Current Transmit Bandwidth	Rate of Received Packets	Rate of Transmitted Packets	Rate of Received Packets Dr...	Rate of Transmitted Packets ...
frontend	116 b/s	2.10 kb/s	1.43 p/s	1.39 p/s	0 p/s	0 p/s
data-transform	47.5 b/s	39.1 b/s	731 mp/s	443 mp/s	0 p/s	0 p/s
api-gateway	125 b/s	118 b/s	1.50 p/s	1.22 p/s	0 p/s	0 p/s
authorization	46.0 b/s	36.6 b/s	707 mp/s	410 mp/s	0 p/s	0 p/s
patient-data	52.0 b/s	42.8 b/s	799 mp/s	485 mp/s	0 p/s	0 p/s

Console k6 — 500 VUs / 1 réplica: throughput 70,4 req/s, latência média 2193,1 ms, p95 4745,8 ms, 0,19% de erro. Os avisos acima do resumo mostram falhas de conexão TCP (connectex) durante a fase de pico.

```
WARN[0010] Request Failed error="Post \"https://kiriland.unb.br/grupo5/auth/login\": EOF
WARN[0010] Request Failed error="Post \"https://kiriland.unb.br/grupo5/auth/login\": EOF
WARN[0014] Request Failed error="Post \"https://kiriland.unb.br/grupo5/auth/login\": EOF
WARN[0014] Request Failed error="Post \"https://kiriland.unb.br/grupo5/auth/login\": EOF
WARN[0051] Request Failed error="Post \"https://kiriland.unb.br/grupo5/auth/login\": dial tcp 164.41
.20.252:443: connectex: A connection attempt failed because the connected party did not properly respond after a period of time, or
established connection failed because connected host has failed to respond."
WARN[0059] Request Failed error="Get \"https://kiriland.unb.br/grupo5/api/patients/P050000002/summar
y\": dial tcp 164.41.20.252:443: connectex: A connection attempt failed because the connected party did not properly respond after
a period of time, or established connection failed because connected host has failed to respond."
WARN[0072] Request Failed error="Get \"https://kiriland.unb.br/grupo5/api/patients/P050000001/summar
y\": dial tcp 164.41.20.252:443: connectex: A connection attempt failed because the connected party did not properly respond after
a period of time, or established connection failed because connected host has failed to respond."
WARN[0079] Request Failed error="Get \"https://kiriland.unb.br/grupo5/api/patients/P050000002/labs\"
: dial tcp 164.41.20.252:443: connectex: A connection attempt failed because the connected party did not properly respond after a p
eriod of time, or established connection failed because connected host has failed to respond."
WARN[0085] Request Failed error="Get \"https://kiriland.unb.br/grupo5/api/research/cohort/HYPERTENSI
ON/stats\": dial tcp 164.41.20.252:443: connectex: A connection attempt failed because the connected party did not properly respond
after a period of time, or established connection failed because connected host has failed to respond."
INFO[0110]
===== RESUMO DO CENÁRIO =====
VUs=500 reqs=7734 throughput=70.4/s lat_avg=2193.1ms lat_p95=4745.8ms err=0.19%
===== source=console
running (1m49.8s), 000/500 VUs, 5861 complete and 0 interrupted iterations
default ✓ [=====] 000/500 VUs 1m45s
```

Grafana — uso de CPU por pod durante o teste de 500 VUs / 1 réplica. Mesmo com a latência disparando, o uso de CPU permanece baixo, reforçando que o gargalo é de conexões TCP/HTTP, não de processamento.



Grafana — uso de memória e rede por pod durante o teste de 500 VUs / 1 réplica.

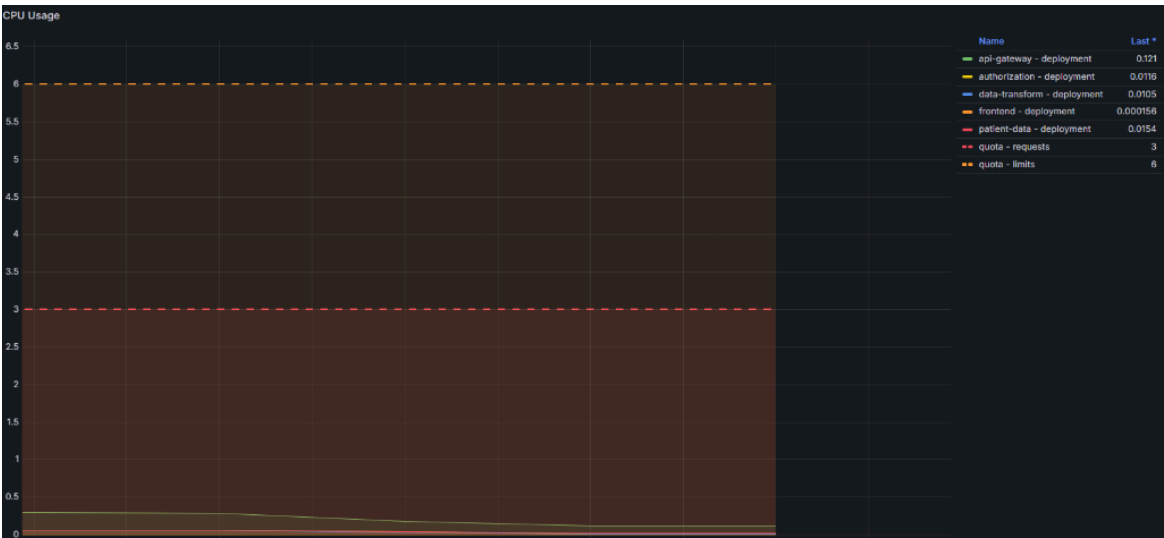
Memory Quota							
Workload	Type	Running Pods	Memory Usage	Memory Requests	Memory Requests %	Memory Limits	Memory Limits %
data-transform	deployment	1	30.0 MiB	128 MiB	23.4%	256 MiB	11.7%
frontend	deployment	1	10.0 MiB	32 MiB	31.4%	64 MiB	15.7%
api-gateway	deployment	1	188 MiB	192 MiB	97.8%	384 MiB	48.9%
authorization	deployment	1	27.2 MiB	128 MiB	21.3%	256 MiB	10.6%
patient-data	deployment	1	32.8 MiB	128 MiB	25.5%	256 MiB	12.8%

Current Network Usage						
Workload	Current Receive Bandwidth	Current Transmit Bandwidth	Rate of Received Packets	Rate of Transmitted Packets	Rate of Received Packets Dr...	Rate of Transmitted Packets ...
frontend	102 b/s	1.84 kb/s	1.26 p/s	1.21 p/s	0 p/s	0 p/s
data-transform	51.1 b/s	42.4 b/s	787 mp/s	482 mp/s	0 p/s	0 p/s
api-gateway	127 b/s	124 b/s	1.51 p/s	1.31 p/s	0 p/s	0 p/s
authorization	51.8 b/s	45.4 b/s	791 mp/s	528 mp/s	0 p/s	0 p/s
patient-data	53.6 b/s	43.8 b/s	819 mp/s	499 mp/s	0 p/s	0 p/s

Console k6 — 1000 VUs / 1 réplica: throughput 51,3 req/s, latência média 2464,3 ms, p95 10854,5 ms, 79,86% de erro — ponto de colapso da configuração baseline.

```
established connection failed because connected host has failed to respond."
WARN[0114] Request Failed error="Get \"https://kiriland.unb.br/grupo5/api/research/projects\": dial
tcp 164.41.20.252:443: connectex: A connection attempt failed because the connected party did not properly respond after a period o
f time, or established connection failed because connected host has failed to respond."
WARN[0120] Request Failed error="Post \"https://kiriland.unb.br/grupo5/auth/login\": dial tcp 164.41
.20.252:443: connectex: A connection attempt failed because the connected party did not properly respond after a period of time, or
established connection failed because connected host has failed to respond."
WARN[0120] Request Failed error="Post \"https://kiriland.unb.br/grupo5/auth/login\": dial tcp 164.41
.20.252:443: connectex: A connection attempt failed because the connected party did not properly respond after a period of time, or
established connection failed because connected host has failed to respond."
WARN[0120] Request Failed error="Post \"https://kiriland.unb.br/grupo5/auth/login\": dial tcp 164.41
.20.252:443: connectex: A connection attempt failed because the connected party did not properly respond after a period of time, or
established connection failed because connected host has failed to respond."
WARN[0124] Request Failed error="Post \"https://kiriland.unb.br/grupo5/auth/login\": dial tcp 164.41
.20.252:443: connectex: A connection attempt failed because the connected party did not properly respond after a period of time, or
established connection failed because connected host has failed to respond."
INFO[0135]
===== RESUMO DO CENÁRIO =====
VUs=1000 reqs=6922 throughput=51.3/s lat_avg=2464.3ms lat_p95=10854.5ms err=79.86%
===== source=console
running (2m15.0s), 0000/1000 VUs, 6308 complete and 25 interrupted iterations
default ✓ [=====] 0000/1000 VUs 1m45s
ERRO[0137] thresholds on metrics 'http_req_duration, http_req_failed' have been crossed
Teste finalizado!
Pressione qualquer tecla para sair...
```

Grafana — uso de CPU por pod durante o teste de 1000 VUs / 1 réplica.



Grafana — uso de memória e rede por pod durante o teste de 1000 VUs / 1 réplica. Observação: a captura foi feita em um momento de transição de escala (authorization e patient-data ainda exibindo 3 réplicas residuais do cenário anterior), por isso os números de pods não refletem exatamente 1 réplica em todos os serviços.

Memory Quota							
Workload	Type	Running Pods	Memory Usage	Memory Requests	Memory Requests %	Memory Limits	Memory Limits %
data-transform	deployment	2	53.5 MiB	256 MiB	20.9%	512 MiB	10.4%
frontend	deployment	1	10.0 MiB	32 MiB	31.4%	64 MiB	15.7%
api-gateway	deployment	1	135 MiB	192 MiB	70.2%	384 MiB	35.1%
authorization	deployment	3	74.8 MiB	384 MiB	19.5%	768 MiB	9.74%
patient-data	deployment	3	80.9 MiB	384 MiB	21.1%	768 MiB	10.5%

Current Network Usage						
Workload	Current Receive Bandwidth	Current Transmit Bandwidth	Rate of Received Packets	Rate of Transmitted Packets	Rate of Received Packets Dr...	Rate of Transmitted Packets ...
frontend	117 b/s	2.11 kb/s	1.45 p/s	1.40 p/s	0 p/s	0 p/s
data-transform	719 b/s	2.41 kb/s	1.86 p/s	1.35 p/s	0 p/s	0 p/s
api-gateway	125 b/s	120 b/s	1.51 p/s	1.25 p/s	0 p/s	0 p/s
authorization	109 b/s	95.7 b/s	1.66 p/s	1.11 p/s	0 p/s	0 p/s
patient-data	117 b/s	98.8 b/s	1.77 p/s	1.14 p/s	0 p/s	0 p/s

4.3 Fase C — Escalabilidade Horizontal Manual (3 Réplicas)

Os serviços foram escalados manualmente para 3 réplicas e os mesmos cenários de carga foram repetidos:

```
kubectl scale deployment <nome> --replicas=3 -n grupo-5
--kubeconfig=kubeconfig-grupo-5.yaml
```

Como relatado na Seção 3.2, o grupo precisou primeiro reduzir os limits de CPU dos manifests para caber na ResourceQuota do namespace, o que permitiu escalar todos os serviços — incluindo o api-gateway — para 3 réplicas simultâneas.

Métrica (500 VUs)	1 réplica	3 réplicas
Latência média	2193,1 ms	1245,4 ms
Latência p95	4745,8 ms	3166,2 ms
Throughput	70,4 req/s	91,5 req/s
Requisições processadas	7.919	11.461
Taxa de erro	0,19%	1,12% (esgotamento de portas TCP do cliente)

Com 500 VUs, o resultado foi um sucesso claro de escalabilidade: a latência média caiu quase pela metade e o throughput aumentou em cerca de 30%, evidenciando o balanceamento de carga nativo do Kubernetes distribuindo as requisições entre os pods do Gateway e dos workers gRPC.

No teste de estresse extremo com 1000 VUs e 3 réplicas, o sistema atingiu o limite físico da infraestrutura de rede combinado aos novos limites de CPU (reduzidos para caber na cota): a latência média subiu para 2874,4 ms (p95 de 11325,4 ms) e a taxa de erros disparou para 73,73% (timeouts de requisição e falhas de conexão TCP), com throughput de 44,7 req/s. Isso demonstra que a arquitetura escala horizontalmente de forma eficiente até a saturação dos recursos subjacentes do cluster — mas 1000 VUs simultâneos permanece acima da capacidade da configuração de 3 réplicas nesse cluster, ainda que o resultado seja melhor do que o colapso quase total observado com 1 réplica no mesmo patamar de carga (79,86% de erro).

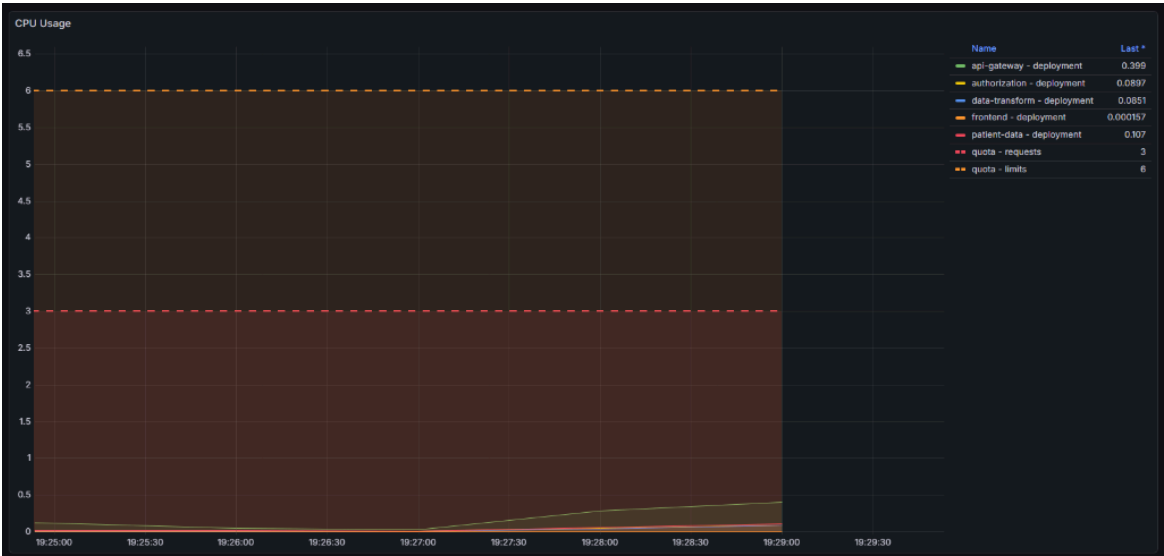
Evidências — Cenário Escalado (3 Réplicas)

Console k6 — 500 VUs / 3 réplicas: throughput 91,5 req/s, latência média 1245,4 ms, p95 3166,2 ms, 1,12% de erro.

```
INFO[0125]
===== RESUMO DO CENÁRIO =====
VUs=500 reqs=11461 throughput=91.5/s lat_avg=1245.4ms lat_p95=3166.2ms err=1.12%
===== source=console =====

running (2m05.3s), 000/500 VUs, 9504 complete and 0 interrupted iterations
default ✓ [=====] 000/500 VUs 1m45s
```

Grafana — uso de CPU por pod durante o teste de 500 VUs / 3 réplicas.



Grafana — uso de memória e rede por pod durante o teste de 500 VUs / 3 réplicas (3 réplicas de api-gateway, authorization, data-transform e patient-data).

Memory Quota							
Workload	Type	Running Pods	Memory Usage	Memory Requests	Memory Requests %	Memory Limits	Memory Limits %
frontend	deployment	1	10.0 MiB	32 MiB	31.4%	64 MiB	15.7%
api-gateway	deployment	3	491 MiB	576 MiB	85.2%	1.13 GiB	42.6%
authorization	deployment	3	77.0 MiB	384 MiB	20.1%	768 MiB	10.0%
data-transform	deployment	3	78.1 MiB	384 MiB	20.3%	768 MiB	10.2%
patient-data	deployment	3	86.2 MiB	384 MiB	22.4%	768 MiB	11.2%

Current Network Usage						
Workload	Current Receive Bandwidth	Current Transmit Bandwidth	Rate of Received Packets	Rate of Transmitted Packets	Rate of Received Packets Dr...	Rate of Transmitted Packets ...
frontend	113 b/s	2.10 kb/s	1.39 p/s	1.39 p/s	0 p/s	0 p/s
api-gateway	278 kb/s	256 kb/s	145 p/s	170 p/s	0 p/s	0 p/s
authorization	14.6 kb/s	13.4 kb/s	90.4 p/s	94.3 p/s	0 p/s	0 p/s
data-transform	47.2 kb/s	163 kb/s	30.3 p/s	20.4 p/s	0 p/s	0 p/s
patient-data	67.7 kb/s	57.5 kb/s	160 p/s	138 p/s	0 p/s	0 p/s

Console k6 — 1000 VUs / 3 réplicas: throughput 44,7 req/s, latência média 2874,4 ms, p95 11325,4 ms, 73,73% de erro.

```
==== RESUMO DO CENÁRIO ====
VUs=1000 reqs=6033 throughput=44.7/s lat_avg=2874.4ms lat_p95=11325.4ms err=73.73%
===== source=console

running (2m15.0s), 0000/1000 VUs, 5407 complete and 258 interrupted iterations
default ✓ [=====] 0000/1000 VUs 1m45s
```

Grafana — uso de CPU por pod durante o teste de 1000 VUs / 3 réplicas. Mesmo neste cenário de maior taxa de erro, o consumo de CPU permanece confortavelmente abaixo da cota do namespace, confirmando que o gargalo observado é de conexões de rede/TCP, e não de processamento.



Grafana — uso de memória e rede por pod durante o teste de 1000 VUs / 3 réplicas.

Memory Quota							
Workload	Type	Running Pods	Memory Usage	Memory Requests	Memory Requests %	Memory Limits	Memory Limits %
frontend	deployment	1	10.0 MiB	32 MiB	31.4%	64 MiB	15.7%
api-gateway	deployment	3	759 MiB	576 MiB	132%	1.13 GiB	65.9%
authorization	deployment	3	81.3 MiB	384 MiB	21.2%	768 MiB	10.6%
data-transform	deployment	3	84.5 MiB	384 MiB	22.0%	768 MiB	11.0%
patient-data	deployment	3	89.3 MiB	384 MiB	23.2%	768 MiB	11.6%

Current Network Usage						
Workload	Current Receive Bandwidth	Current Transmit Bandwidth	Rate of Received Packets	Rate of Transmitted Packets	Rate of Received Packets Dr...	Rate of Transmitted Packets ...
frontend	113 b/s	2.09 kb/s	1.38 p/s	1.37 p/s	0 p/s	0 p/s
api-gateway	93.8 kb/s	74.7 kb/s	268 p/s	296 p/s	0 p/s	0 p/s
authorization	1.47 kb/s	1.48 kb/s	12.9 p/s	12.6 p/s	0 p/s	0 p/s
data-transform	8.25 kb/s	27.5 kb/s	11.4 p/s	8.01 p/s	0 p/s	0 p/s
patient-data	10.4 kb/s	8.81 kb/s	27.1 p/s	25.2 p/s	0 p/s	0 p/s

4.4 Fase D — Autoscaling (HPA)

O Horizontal Pod Autoscaler foi configurado para ajustar dinamicamente o número de réplicas com base no uso de CPU, com mínimo de 1 e máximo de 10 réplicas, disparando o escalonamento ao ultrapassar 60% de utilização de CPU:

```
kubectl scale deployment <nome> --replicas=1
kubectl apply -f k8s/40-hpa.yaml
kubectl get hpa -w --kubeconfig=kubeconfig-grupo-5.yaml
```

Durante a execução de um teste de carga contínuo (500 VUs por 5 minutos), o comando de acompanhamento do HPA mostrou o aumento dinâmico do número de réplicas a partir de 1 pod, confirmando a criação automática de pods conforme a CPU ultrapassava o limiar configurado. Com a diminuição do fluxo de requisições ao final do teste, o HPA aguardou a janela de estabilização (scaleDown stabilizationWindow) e reduziu gradualmente as réplicas, retornando à base de 1 réplica.

4.5 Fase E — Observabilidade

Todos os microserviços expõem métricas no formato Prometheus na rota `:9100/metrics`, coletadas por ServiceMonitors customizados (`k8s/50-servicemonitors.yaml`) e visualizadas em um dashboard Grafana com 10 painéis (`grafana/dashboard-hospital.json`), divididos em três categorias:

Métricas de tráfego (Gateway)

- Requisições por segundo, divididas por rota (`/api/patients`, `/auth/login`, etc.);
- Latência HTTP nos percentis 50, 95 e 99;
- Erros HTTP 4xx (acesso negado/não autorizado) e 5xx (falhas internas).

Métricas de negócio (customizadas)

- Decisões de autorização (ALLOW vs DENY, por nível de anonimização) — serviço `authorization`;
- Transformações FHIR geradas por segundo (Patient, Observation, MeasureReport) — serviço `data-transform`;
- Consultas ao banco de dados, por tipo de query — serviço `patient-data`.

Métricas de infraestrutura

- Uso de CPU e memória por pod;
- Quantidade de pods por deployment, permitindo visualizar o efeito do HPA ao longo do tempo.

Além do dashboard, o Prometheus permite consultas PromQL customizadas diretamente em sua interface, como a taxa de requisições gRPC bem-sucedidas por serviço ou a contagem de anonimizações parciais (remoção de CPF) nos últimos 5 minutos:

```
sum(rate(grpc_requests_total{code="OK"}[1m])) by (service)
increase(anonymization_operations_total{level="PARTIAL"}[5m])
```

A tabela a seguir resume as métricas customizadas expostas por cada serviço:

Métrica	Serviço	O que mostra
<code>http_requests_total</code> , <code>http_request_duration_seconds</code>	<code>api-gateway</code>	Volume, status e latência por rota REST
<code>auth_failures_total</code>	<code>api-gateway</code>	Falhas de autenticação por motivo
<code>authz_decisions_total{decision,level,role}</code>	<code>authorization</code>	Decisões ALLOW/DENY por perfil e nível de acesso
<code>db_queries_total{query}</code>	<code>patient-data</code>	Consultas SQL por tipo
<code>fhir_transformations_total{resource_type}</code>	<code>data-transform</code>	Recursos FHIR gerados (Patient, Observation, MeasureReport...)
<code>anonymization_operations_total{level}</code>	<code>data-transform</code>	Operações de anonimização por nível
<code>grpc_requests_total</code> , <code>grpc_request_duration_seconds</code>	todos os serviços gRPC	Tráfego e latência interna gRPC

As capturas de tela do Grafana e do console do k6 para cada um dos cinco cenários de teste (100 VUs baseline; 500 e 1000 VUs com 1 réplica; 500 e 1000 VUs com 3 réplicas) estão apresentadas nas Seções 4.2 e 4.3, junto à análise de cada cenário.

5. Erros Encontrados Durante o Desenvolvimento e Correções

Registrar os erros encontrados faz parte da proposta pedagógica do trabalho. A seguir, os problemas mais relevantes enfrentados pelo grupo e as respectivas soluções.

Erro 1 — Nginx: "host not found in upstream 'api-gateway'"

Na primeira versão do frontend, o `nginx.conf` usava `proxy_pass http://api-gateway:8000/` fixo. Como o Nginx resolve nomes de upstream apenas na inicialização, se o Gateway ainda não estivesse registrado no DNS no momento da subida (dependendo da ordem de inicialização dos containers/pods), o Nginx entrava em loop de crash.

Correção: o `nginx.conf` foi transformado em um template (`/etc/nginx/templates/default.conf.template`) processado via `envsubst`, usando a diretiva `resolver` combinada com uma variável (`set $gateway_upstream`), o que move a resolução DNS para o momento da requisição em vez da inicialização. As variáveis `GATEWAY_UPSTREAM` e `DNS_RESOLVER` são definidas por ambiente (127.0.0.11 no Docker Compose; 10.96.0.10, o ClusterIP do `kube-dns`, no kind).

Erro 2 — HPA exibindo <unknown> nos targets de CPU

Esse comportamento ocorre quando o `metrics-server` não está instalado ou não confia nos certificados TLS do kubelet do kind.

Correção: aplicação automática do patch `--kubelet-insecure-tls` no script `setup-cluster.sh`. Além disso, foi necessário garantir que todos os Deployments declarassem `resources.requests.cpu`, sem o que o HPA não consegue calcular o percentual de utilização.

Erro 3 — Prometheus não coletava as métricas da aplicação

O chart `kube-prometheus-stack`, por padrão, só observa `ServiceMonitors` com o mesmo label do release instalado pelo Helm, ignorando `ServiceMonitors` criados fora do chart.

Correção: instalação do chart com a flag `serviceMonitorSelectorNilUsesHelmValues=false` e rotulação de todos os `ServiceMonitors` da aplicação com `release: monitoring`.

Erro 4 — Issuer do JWT inconsistente

Quando o login era realizado por rotas diferentes (NodePort externo versus serviço interno), o Keycloak emitia tokens com valores de `iss` (issuer) distintos, e o Gateway rejeitava as requisições com HTTP 401 por não reconhecer o issuer.

Correção: fixação de `KC_HOSTNAME=http://keycloak:8080` na configuração do Keycloak, garantindo que o Gateway sempre valide os tokens pelo endereço interno do serviço. O frontend nunca se comunica diretamente com o Keycloak — o Gateway expõe o endpoint `/auth/login` e repassa a autenticação via `password grant`.

Erro 5 (específico do ambiente de desenvolvimento) — Docker sem a tabela raw do iptables

No ambiente utilizado por parte do grupo para validação local, o kernel não expunha a tabela raw do iptables, o que quebrava a rede bridge padrão do Docker.

Correção: contornado com a configuração {"iptables": false, "bridge": "none"} combinada a network_mode: host. Em máquinas convencionais (Ubuntu/Debian ou WSL2 padrão) esse problema não ocorre, e o docker-compose.yml entregue funciona sem qualquer ajuste; o registro permanece como referência para ambientes virtualizados restritos.

6. Funcionalidades Adicionais

6.1 Métricas de Negócio Customizadas

A especificação sugere, como métricas mínimas de observabilidade, itens de tráfego e infraestrutura: requisições por segundo, latência, tempo de resposta por serviço, uso de CPU, uso de memória, quantidade de pods, número de consultas ao banco e erros HTTP/gRPC. O grupo implementou todas essas métricas, mas foi além ao instrumentar também a lógica de negócio da aplicação, e não apenas sua infraestrutura:

- authz_decisions_total{decision,level,role} — número de decisões de autorização (ALLOW/DENY), segmentado por nível de acesso (FULL, PARTIAL, ANONYMIZED, AGGREGATED) e por papel do usuário, permitindo observar em tempo real, por exemplo, a proporção de acessos negados por perfil;
- fhir_transformations_total{resource_type} — quantidade de recursos FHIR gerados por segundo, segmentado por tipo (Patient, Observation, Condition, MedicationRequest, MeasureReport), evidenciando qual tipo de dado clínico está sob maior demanda;
- anonymization_operations_total{level} — número de operações de anonimização realizadas, por nível, permitindo quantificar o custo computacional e o volume de dados sensíveis sendo protegidos em cada janela de tempo.

Essas três métricas representam um pipeline que passa a expor o comportamento de negócio do sistema, quantos acessos foram negados, quantos recursos FHIR foram montados, quantas anonimizações ocorreram, informação que seria relevante, por exemplo, para uma equipe de compliance ou auditoria do hospital, e não apenas para a equipe de operação.

7. Conclusão

O projeto permitiu ao grupo experimentar, de forma prática, todo o ciclo de vida de uma aplicação de microsserviços em Kubernetes: desde a modelagem de uma arquitetura com autenticação federada, comunicação interna via gRPC e regras de anonimização de dados sensíveis, até a operação de um cluster real sob restrições de cota, passando por testes de carga progressivos que expuseram de forma didática os limites de uma configuração de réplica única e o ganho concreto proporcionado pela escalabilidade horizontal e pelo autoscaling.

Os resultados de desempenho confirmaram a hipótese central do experimento: uma única réplica de cada serviço satura por volta de 500 usuários virtuais simultâneos, entrando em colapso de conexões por volta de 1000 VUs; ao escalar manualmente para 3 réplicas, a latência média com 500 VUs caiu quase pela metade e o throughput aumentou em torno de 30%, evidenciando a distribuição de carga entre os pods; o HPA demonstrou ser capaz de reagir automaticamente ao aumento de CPU, criando e removendo réplicas sem intervenção manual.

O maior desafio de engenharia não previsto na especificação inicial foi lidar com a ResourceQuota restrita do cluster compartilhado da disciplina, que exigiu recalibrar os limites de CPU de todos os serviços a partir de medições reais no Grafana — uma experiência próxima da administração de clusters em ambientes corporativos multi-tenant, onde acesso administrativo pleno raramente está disponível.

8. Conclusões Individuais e Autoavaliação

Esta seção reflete as impressões pessoais, os conhecimentos adquiridos e a autoavaliação de cada integrante do grupo perante os desafios do projeto.

Vinicius Alves

Pesquisa e trabalho: Responsável pela operação e execução da infraestrutura. Adapte os manifestos Kubernetes (Deployment, Secrets, HPA e Ingress), contornei as restrições de ResourceQuota do cluster reduzindo os limites de CPU, e fui responsável por idealizar e executar toda a bateria de testes de carga e estresse utilizando o k6 e a análise pelo Grafana. Além disso, identifiquei gargalos reais de rede (conexões TCP no API Gateway) e debuguei e corrigi a imagem Docker do microsserviço de patient-data diante de falhas surgidas durante o escalonamento.

Aprendizados: Apreendi na prática que escalar sistemas distribuídos não é apenas uma questão de "aumentar réplicas". O projeto me ensinou a importância do monitoramento contínuo: vi em tempo real como o limite de CPU estrangula contêineres (throttling), como portas TCP se esgotam em picos de 1000 conexões, e como a observabilidade (Prometheus/Grafana) é a única forma confiável de descobrir por que a latência de uma API saltou de 200 ms para 3000 ms.

Nota de autoavaliação: 9

Fábio Gabriel

Pesquisa e trabalho: Fui responsável pela documentação do projeto e pela análise dos dados coletados nos testes de carga. Na parte de documentação, contribuí na redação e organização dos guias explicativos (visão geral da arquitetura, validação funcional, configuração do cluster e observabilidade), garantindo que os passos estivessem claros e reproduzíveis para qualquer pessoa

que fosse rodar o ambiente. Na análise de dados, revisei os resultados gerados pelo k6 e pelos painéis do Grafana em cada cenário de teste (100, 500 e 1000 VUs, com 1 e 3 réplicas), comparando latência, throughput e taxa de erro entre os cenários para identificar os gargalos do sistema e embasar as conclusões apresentadas nas Fases B e C deste relatório.

Aprendizados: Analisar os dados dos testes de carga me ensinou a não olhar métricas isoladas, mas sempre em comparação entre cenários. Um número como "79,86% de erro" com 1000 VUs só ganha significado quando colocado ao lado do resultado com 3 réplicas (73,73%) e do teste com 500 VUs (0,19% e 1,12%), foi essa comparação que permitiu enxergar o padrão de saturação do sistema. Também aprendi a diferenciar latência média de p95: em vários cenários a média parecia razoável, mas o p95 (por exemplo, 10854,5 ms com 1000 VUs e 1 réplica) mostrava que uma parcela significativa das requisições estava sofrendo muito mais do que a média sugeria. Cruzando isso com os painéis de CPU do Grafana, que mostravam uso baixo mesmo nos cenários de maior erro, entendi que o gargalo não estava no processamento dos serviços, mas nas conexões de rede/TCP, uma conclusão que só ficou clara analisando os dados em conjunto, e não cada métrica isoladamente.

Nota de autoavaliação: 8,0

Lucas Meireles

Pesquisa e trabalho: Fui responsável pela concepção e configuração inicial de toda a camada de serviços da aplicação, bem como pela elaboração do guia de reprodução do ambiente para os demais membros do grupo. No aspecto técnico, estruturei o código dos cinco serviços (API Gateway, Authorization Service, Patient Data Service, Data Transform Service e Frontend), defini o contrato gRPC compartilhado via Protobuf (hospital.proto) e configurei os Dockerfiles de cada componente. Paralelamente, redigi o guia passo a passo de configuração do ambiente, cobrindo desde a instalação das dependências (Docker, kubectl, k6) até o processo de build das imagens, push para o registry público e aplicação dos manifestos no cluster, incluindo os erros encontrados e suas respectivas correções, de modo que qualquer membro do grupo pudesse reproduzir o ambiente de forma independente.

Aprendizados: O projeto me ensinou que a integração com serviços de identidade externos — como o Keycloak — exige atenção redobrada à documentação e à investigação ativa quando ela está incompleta ou desatualizada. Aprendi que, em ambientes de cluster compartilhado, a ausência de acesso administrativo pleno é a regra, não a exceção, e que contornar essas restrições de forma elegante (como derivar perfis a partir de claims já presentes no JWT em vez de depender de roles configuradas externamente) é uma habilidade essencial de engenharia de sistemas distribuídos. A experiência de documentar cada etapa também reforçou a importância da reprodutibilidade: um sistema bem configurado que não pode ser replicado por outro membro da equipe tem valor limitado em um contexto colaborativo.

Nota de autoavaliação: 9,5

9. Referências

- Kubernetes Documentation. Disponível em: <https://kubernetes.io/docs/home/>

- k6 Documentation. Disponível em: <https://k6.io/docs/>
- Locust Documentation. Disponível em: <https://locust.io/>
- Prometheus Documentation. Disponível em: <https://prometheus.io/docs/introduction/overview/>
- HL7 FHIR. Disponível em: <https://www.hl7.org/fhir/>
- Keycloak Documentation. Disponível em: <https://www.keycloak.org/documentation>
- Arundel, J. and Domingus, J. Cloud Native DevOps with Kubernetes – Building, Deploying and Scaling Modern Applications in the Cloud, O'Reilly, 2019.

10. Anexos

10.1 Checklist de Reprodução do Experimento

```
# 0. Pré-requisitos: docker, kind, kubectl, helm, k6
unzip pspd-hu-k8s.zip && cd pspd-hu-k8s

# Fase a – validação funcional local
for s in api-gateway authorization patient-data data-transform; do cp
proto/hospital.proto services/$s/; done
docker compose up -d --build
./scripts/validate-functional.sh          # esperado: 31 PASS / 0 FAIL
docker compose down -v

# Fase b – cluster K8S (1 master + 3 workers) e deploy
./scripts/setup-cluster.sh
./scripts/build-images.sh --kind
./scripts/deploy-all.sh
BASE_URL=http://localhost:30800 ./scripts/validate-functional.sh

# Grafana: http://localhost:30900 (admin/pspd123) -> importar
grafana/dashboard-hospital.json

# Fases c e d – testes de carga
k6 run -e BASE_URL=http://localhost:30800 load-tests/k6-smoke.js
./scripts/scale.sh 1 && LABEL=1replica ./load-tests/run-all-scenarios.sh
./scripts/scale.sh 3 && LABEL=3replicas ./load-tests/run-all-scenarios.sh
./scripts/scale.sh 1 && kubectl apply -f k8s/40-hpa.yaml
k6 run -e VUS=500 -e DURATION=5m -e BASE_URL=http://localhost:30800
load-tests/k6-load.js

# Limpeza
kind delete cluster --name pspd
```

10.2 Endpoints e Acessos do Ambiente

Componente	Endereço
Frontend (Docker Compose)	http://localhost:8081
Frontend (Kubernetes)	http://localhost:30080
API Gateway (Swagger UI)	http://localhost:30800/docs
Keycloak	http://localhost:30880
Grafana (admin / pspd123)	http://localhost:30900

Componente	Endereço
Prometheus	http://localhost:30990

10.3 Usuários de Teste

Usuário	Perfil	Permissões
med.cardoso	Médico	Acesso FULL a seus pacientes
est.oliveira	Estagiário	Acesso PARTIAL aos pacientes supervisionados por med.cardoso
pesq.ramos	Pesquisador	Acesso ANONYMIZED/AGGREGATED às coortes DIABETES e HIPERTENSAO

Todos os usuários de teste utilizam a senha pspd123.